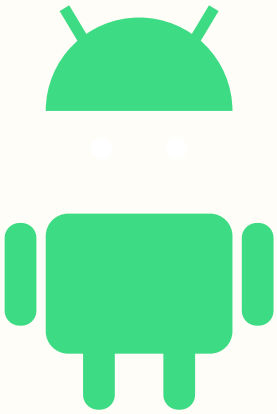
A brain-friendly guide for Android developers

Flutter Core Concepts for Android Brains

Widgets, layouts, lists, navigation, state, async, isolates, channels and animations - each one next to the Android idea you already own.



what you know





what you'll learn

INSIDE:

- Declarative UI: UI = f(state)
- Layouts & the constraint rule
- RecyclerView → ListView.builder
- Intents → Navigator
- ViewModel → Provider
- Coroutines → Future/Stream, threads → isolates
- Platform channels, animations, theming, the three trees

Stop mutating views. Start describing UI. Your life gets simpler.

What's in this doc

1 **Everything is a Widget**
Declarative UI and why you never call setText() again

2 **Layouts**
Constraints go down, sizes go up, parent sets position

3 **Lists & Grids**
RecyclerView, minus the ceremony

4 **Navigation**
Activities & Intents become routes & a Navigator

5 **State Management**
Your ViewModel has a new name

6 **Async, Isolates & Background Work**
Coroutines, Flows and threads - Dart style

7 **Talking to Android: Platform Channels**
When Dart needs Kotlin

8 **Animations**
Implicit, explicit, and Hero

9 **Theming & Resources**
Where did res/values go?

10 **Under the Hood**
Widget, Element & RenderObject trees, Keys and BuildContext

Each chapter matches a concept in the app's LEARN tab. Read here, then open the app and hit the "Animate" tab to watch it happen.

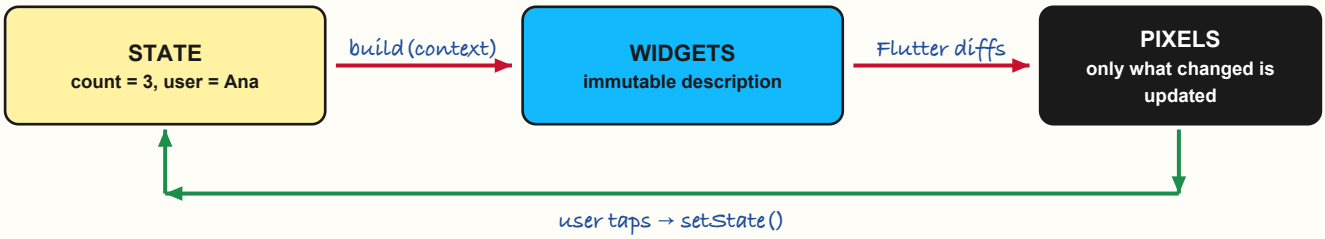
CHAPTER 1

Everything is a Widget

Declarative UI - and why you never call setText() again

In Android (Views) you write **imperative** UI code: find a view, then tell it to change. Flutter is **declarative**: you write a function that says what the UI *should look like for the current state*. When the state changes, Flutter calls that function again and works out the difference itself.

UI = f(state)



ANDROID (Kotlin)

```
// Imperative: mutate the view
val tv = findViewById<TextView>(R.id.count)
button.setOnClickListener {
    count++
    tv.text = "Count: $count"
    tv.isVisible = count <= 5
}
```

FLUTTER (Dart)

```
// Declarative: describe the result
@override
Widget build(BuildContext context) {
  return Column(children: [
    if (count <= 5) Text('Count: $count'),
    ElevatedButton(
      onPressed: () => setState(() => count++),
      child: const Text('+1'),
    ),
  ]);
}
```

StatelessWidget vs StatefulWidget

Concept	Flutter	Closest Android idea
StatelessWidget	Everything comes from the constructor. A pure function of its inputs.	A custom View whose look depends only on its attributes.
StatefulWidget	Two classes: an immutable widget + a long-lived State<T> object that holds mutable fields and survives rebuilds.	An Activity/Fragment with fields; or a View with internal state.
setState(() {...})	Mark this State dirty → build() runs again on the next frame.	Changing a field and calling invalidate() / requestLayout().

STATEFUL WIDGET

```
class Counter extends StatefulWidget {
  const Counter({super.key});
  @override
  State<Counter> createState() => _CounterState();
}

class _CounterState extends State<Counter> {
  int _count = 0;

  @override
  Widget build(BuildContext context) {
    return TextButton(
      onPressed: () => setState(() => _count++),
      child: Text('Tapped $_count times'),
    );
  }
}
```

← the immutable "config" half

← framework calls this once

← the long-lived half
← mutable field ≈ Activity field

← called MANY times - keep it fast

← change + schedule rebuild
← new Text widget each build



BRAIN POWER

Why does Flutter keep `_count` in a separate State object instead of in the widget itself? (Hint: widgets are recreated on every parent rebuild. What would happen to a field stored in them?)

there are no Dumb Questions

Q: **Isn't creating new widgets on every frame slow?**

A: Widgets are tiny, short-lived Dart objects - cheaper than you think (Dart's GC is tuned for this). The expensive objects that do layout and painting are **reused**. Chapter 10 shows how.

Q: **How do I hide a widget? There's no View.GONE.**

A: Just don't build it: `if (show) MyWidget()`. For INVISIBLE use `Visibility(visible: false, maintainSize: true, ...)` or `Opacity`.

Q: **What about Jetpack Compose?**

A: If you know Compose, you already know Flutter's model! @Composable ≈ build(), remember{} ≈ State fields, recomposition ≈ rebuild. The syntax differs; the idea is identical.



BULLET POINTS

- Widgets are immutable blueprints; build() returns a fresh tree each time.
- Default to StatelessWidget. Use StatefulWidget only for local, changing state.
- setState() = "my data changed, please call build() again".
- Padding, Center, GestureDetector, Theme... everything is a widget.

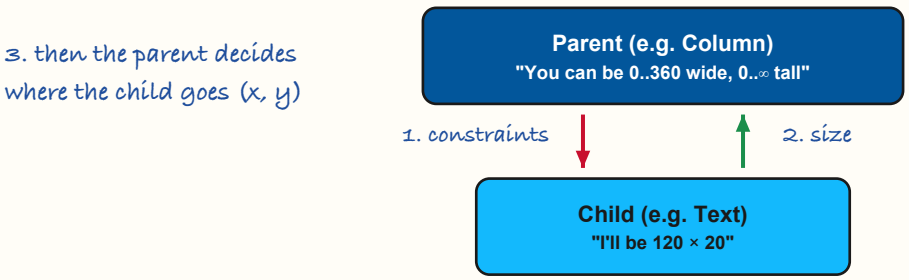
CHAPTER 2

Layouts

Constraints go down. Sizes go up. Parent sets position.

Android measures with `onMeasure(widthSpec, heightSpec)`. Flutter does something very similar, and it fits into one sentence you should tattoo on your arm:

> Constraints go DOWN. Sizes go UP. The parent sets the POSITION.

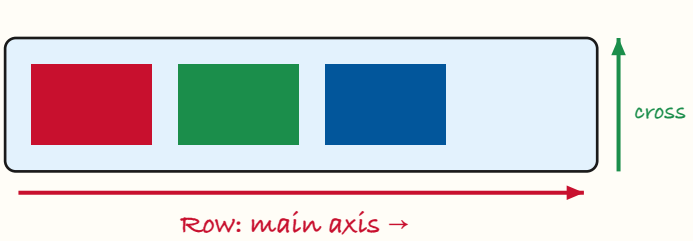


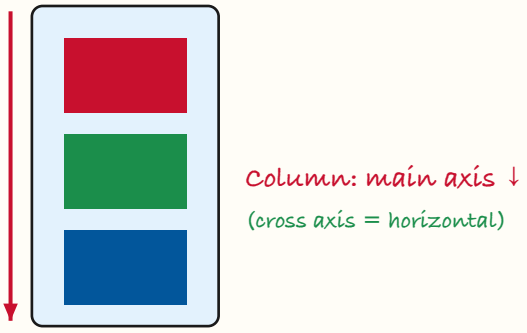
The Android → Flutter layout cheat sheet

Android	Flutter	Tip
LinearLayout horizontal	Row	mainAxisAlignment ≈ gravity along the row
LinearLayout vertical	Column	crossAxisAlignment ≈ gravity across
FrameLayout	Stack (+ Positioned)	Children painted on top of each other
layout_weight	Expanded(flex: n) / Flexible	Takes remaining space by ratio
match_parent	double.infinity / Expanded / CrossAxisAlignment.stretch	
wrap_content	The default for most widgets	Row/Column: mainAxisSize: MainAxisSize.min
android:padding	Padding widget / Container(padding:)	
layout_margin	Padding around the child / SizedBox gaps	No margin attribute on most widgets
gravity / layout_gravity	Align / Center	
ScrollView	SingleChildScrollView	For small content; lists use ListView
FlexboxLayout / ChipGroup	Wrap	
ConstraintLayout	Row + Column + Stack + Expanded; LayoutBuilder	Nest freely - nesting is cheap
layout-sw600dp	LayoutBuilder / MediaQuery.sizeOf(context)	Decide the layout in code
Space / View as spacer	SizedBox(height: 16) / Spacer()	

Android	Flutter	Tip
CardView	Card	

Main axis vs cross axis





ANDROID (XML)

```
<LinearLayout
  android:orientation="horizontal"
  android:gravity="center_vertical">
  <ImageView ... />
  <TextView
    android:layout_width="0dp"
    android:layout_weight="1" />
  <Button ... />
</LinearLayout>
```

FLUTTER (Dart)

```
Row(
  crossAxisAlignment:
      CrossAxisAlignment.center,
  children: [
    const Icon(Icons.star),
    Expanded(          // weight = 1
      child: Text(title),
    ),
    FilledButton(...),
  ],
)
```



Watch it!

The #1 beginner error: **"Vertical viewport was given unbounded height"**. You put a `ListView` (which wants infinite height) inside a `Column` (which gives its children unbounded height). Fix: wrap the ListView in `Expanded` - exactly like giving a RecyclerView `0dp + layout_weight=1` inside a LinearLayout.



SHARPEN YOUR PENCIL

Which widgets would you nest to build this list row? *[avatar] Name (bold) / subtitle underneath ..... [chevron]*

CHAPTER 3

Lists & Grids

RecyclerView, minus the ceremony

RecyclerView needs an Adapter, a ViewHolder, a LayoutManager, and a way to notify changes. `ListView.builder` needs... a count and a function. It still only builds the rows that are visible, and reuses the heavy objects under the hood.

ANDROID (Kotlin)

```
class PostAdapter(
  private val posts: List<Post>
) : RecyclerView.Adapter<PostAdapter.VH>() {

  class VH(val b: ItemPostBinding)
    : RecyclerView.ViewHolder(b.root)

  override fun onCreateViewHolder(
      p: ViewGroup, t: Int) = VH(
    ItemPostBinding.inflate(
      LayoutInflater.from(p.context), p, false))

  override fun onBindViewHolder(h: VH, i: Int) {
    h.b.title.text = posts[i].title
  }

  override fun getItemCount() = posts.size
}

recycler.layoutManager =
    LinearLayoutManager(this)
recycler.adapter = PostAdapter(posts)
```

FLUTTER (Dart)

```
ListView.builder(
  itemCount: posts.length,
  itemBuilder: (context, i) {
    return ListTile(
      title: Text(posts[i].title),
      onTap: () => open(posts[i]),
    );
  },
)

// That's it. Really.
//
// Data changed?
// setState(() => posts = newPosts);
// (no notifyDataSetChanged)
```

Android	Flutter
RecyclerView + LinearLayoutManager	ListView.builder
GridLayoutManager(spanCount = 3)	GridView.builder + SliverGridDelegateWithFixedCrossAxisCount
Adaptive spans	SliverGridDelegateWithMaxCrossAxisExtent
DividerItemDecoration	ListView.separated
Multiple view types	Return different widgets from itemBuilder
notifyItemInserted + animations	AnimatedList
ItemTouchHelper (swipe)	Dismissible
ItemTouchHelper (drag)	ReorderableListView
SwipeRefreshLayout	RefreshIndicator
DiffUtil / stable IDs	Automatic diffing + Key (ValueKey(item.id))
Paging 3	infinite_scroll_pagination package, or load more in itemBuilder
NestedScrollView + CollapsingToolbar	CustomScrollView + SliverAppBar + SliverList



RELAX

Open the app → Learn → "Lists & Grids" → Animate tab. It builds a 10,000-row list and counts how many times itemBuilder actually runs. Scroll and watch - it stays small. That's your RecyclerView recycling, for free.

there are no **Dumb Questions**

Q: **When do I use plain ListView(children: [...]) instead of .builder?**

A: Only for short, fixed lists (a settings screen with 8 rows). It builds every child up front, like a LinearLayout inside a ScrollView.

Q: **Do I need Keys?**

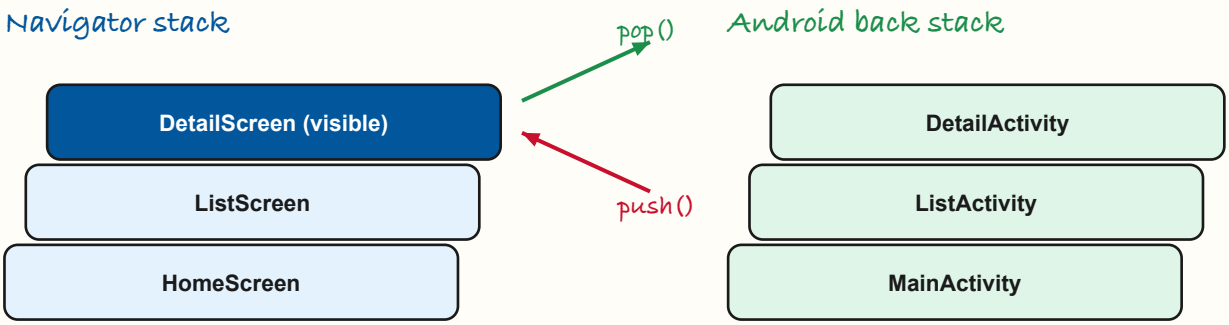
A: When items can be **reordered, inserted or removed** and they hold state (a checkbox, an animation, a text field). Give each a `ValueKey(item.id)` so Flutter matches old and new rows correctly - the same reason you use stable IDs in RecyclerView.

CHAPTER 4

Navigation

Activities & Intents become routes & a Navigator

A **Route** is a screen. The **Navigator** is a stack of routes - exactly like the Activity back stack. `push` ≈ `startActivity`, `pop` ≈ `finish()`, and the system Back button pops automatically.



Android	Flutter
`startActivity(Intent(this, Detail::class.java))`	`Navigator.push(context, MaterialPageRoute(builder: (_) => const Detail()))`
`Intent extras: putExtra("id", 42)`	`Constructor argument: Detail(id: 42) or pushNamed(..., arguments: 42)`
`registerForActivityResult + launch`	`final result = await Navigator.push(...)`
`setResult(RESULT_OK, data); finish()`	`Navigator.pop(context, data)`
`finish()`	`Navigator.pop(context)`
`FLAG_ACTIVITY_CLEAR_TOP / CLEAR_TASK`	`pushAndRemoveUntil / popUntil`
`Manifest <activity> registry`	`MaterialApp(onGenerateRoute: ...) or go_router routes`
`Navigation Component + NavGraph + deep links`	`go_router package (URL-based routes)`
`OnBackPressedCallback`	`PopScope(canPop: ..., onPopInvokedWithResult: ...)`
`Fragment transactions inside one Activity`	`Nested Navigator / IndexedStack for tabs`

lib/features/components/navigation_demo.dart

```
Future<void> _openForResult() async {
  final result = await Navigator.pushNamed<String>(
    context,
    AppRoutes.navigationDetail,
    arguments: _controller.text,
  );
  if (!mounted) return;
  setState(() => _result = result ?? '(no result)');
}

// In screen B:
onPressed: () => Navigator.pop(context, 'Approved'),
```

← async ≈ launching a coroutine
← waits until screen B pops

← route name ≈ Activity class
← ≈ putExtra(...)

← screen might be gone now!
← ≈ onActivityResult

← ≈ setResult + finish()

there are no **Dumb Questions**

Q: **Named routes or MaterialPageRoute?**

A: Both work. This app uses a central `onGenerateRoute` so you can see all screens in one place (like the manifest). For deep links, web URLs and nested navigation, use **go_router**.

Q: **How do I open another app, dial a number or open a URL (implicit Intents)?**

A: Use the `url_launcher` package: `launchUrl(Uri.parse('tel:123'))`. For sharing use `share_plus`. Under the hood they fire Android Intents.

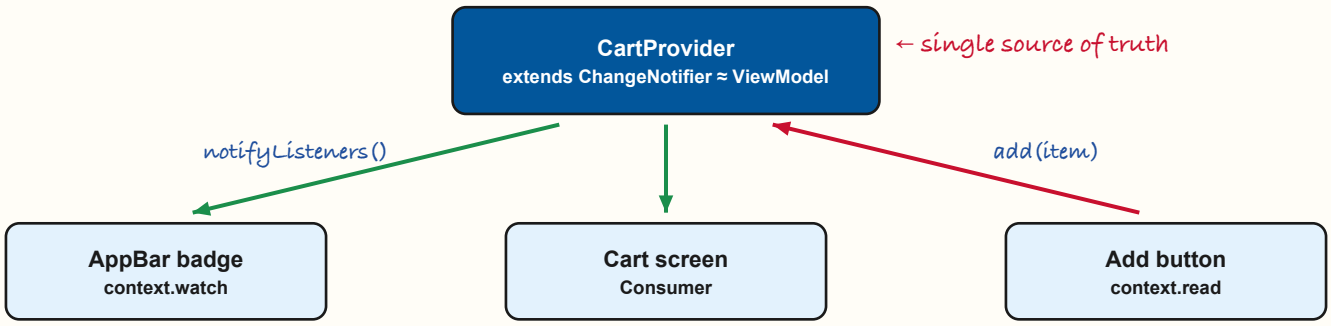
CHAPTER 5

State Management

Your ViewModel has a new name

Flutter splits state into two kinds. Knowing which is which answers 90% of "which state management should I use?" questions.

Kind	Examples	Flutter tool	Android equivalent
Ephemeral (local) state	Current tab, is this checkbox ticked, animation progress, text field contents.	setState() inside a StatefulWidget	Fields in a View / Fragment
App state	Logged-in user, cart, settings, data loaded from the network, anything shared between screens.	A provider: ChangeNotifier / Riverpod / Bloc	ViewModel + LiveData/StateFlow, repositories



ANDROID (Kotlin)

```
class CartViewModel : ViewModel() {
  private val items = mutableListOf<Item>()
  private val _count = MutableLiveData(0)
  val count: LiveData<Int> = _count

  fun add(item: Item) {
    items += item
    _count.value = items.size
  }
}

// Fragment
private val vm: CartViewModel
    by activityViewModels()
vm.count.observe(viewLifecycleOwner) {
  badge.text = "$it"
}
addBtn.setOnClickListener { vm.add(item) }
```

FLUTTER (Dart)

```
class CartProvider extends ChangeNotifier {
  final _items = <CartItem>[];
  int get count => _items.length;

  void add(CartItem item) {
    _items.add(item);
    notifyListeners();   // ≈ setValue
  }
}

// main.dart - provide it high in the tree
ChangeNotifierProvider(
  create: (_) => CartProvider(),
  child: const MyApp(),
)

// Any widget below:
final count =
    context.watch<CartProvider>().count;
onPressed: () =>
    context.read<CartProvider>().add(item)
```

API	Meaning	Android
`context.watch<T>()`	Get T AND rebuild this widget when it notifies	observe()
`context.read<T>()`	Get T once, no rebuild. Use in callbacks (onPressed).	calling vm.doSomething()

API	Meaning	Android
`context.select<T, R>(fn)`	Rebuild only when one field changes	distinctUntilChanged / map
`Consumer<T>(builder:)`	Rebuild only the builder subtree	observing in a small custom View



Watch it!

Never call `context.watch` inside a callback like `onPressed` - use `context.read`. watch is for build(); read is for events. (Flutter will throw an assertion to remind you.)

The other popular options

Library	Character	Feels like
Provider	Simple, official-recommended starting point. Used in this app.	Hilt-lite + ViewModel
Riverpod	Provider's successor: compile-safe, testable, no BuildContext needed.	Hilt + StateFlow
Bloc / Cubit	Events in, states out. Very structured; great for big teams.	MVI + StateFlow
GetX / MobX / Redux	Other styles; less common in new projects.	-

Fireside Chats

Tonight's talk: Do we even need each other?

setState

I'm simple. One line and the widget rebuilds. Why do people install you at all?

"Prop drilling." I've heard the screams.

Fine. But for a checkbox, a tab index or a text field, I'm still the right choice.

Provider

Because you only work inside ONE widget. The moment two screens need the same cart, developers start passing callbacks through six constructors.

I put the data above both screens and let anyone below watch it. Like an activity-scoped ViewModel.

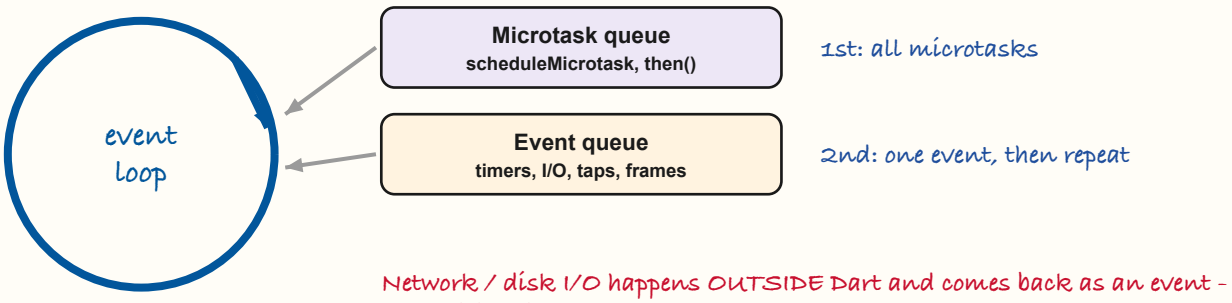
Absolutely. Use you for local state, me for shared state. We're a team.

CHAPTER 6

Async, Isolates & Background Work

Coroutines, Flows and threads - Dart style

Dart runs your code on a **single thread with an event loop** - the same model as Android's main thread and its Looper. `async/await` does not create threads; it pauses your function until a result arrives, letting the loop run other work (like drawing frames) in the meantime. That's exactly how a coroutine on `Dispatchers.Main` behaves.



Network / disk I/O happens OUTSIDE Dart and comes back as an event - so awaiting it never blocks the UI.

Kotlin coroutines	Dart
`suspend fun load(): User`	`Future<User> load() async`
`val u = load() (inside coroutine)`	`final u = await load();`
`launch { ... }`	`Just call an async function (optionally unawaited(...))`
`withContext(Dispatchers.IO) { api() }`	`await api(); (I/O is already non-blocking)`
`withContext(Dispatchers.Default) { crunch() }`	`await Isolate.run(() => crunch());`
`async { } / awaitAll`	`Future.wait([a(), b()])`
`delay(1000)`	`await Future.delayed(const Duration(seconds: 1));`
`Flow<T>`	`Stream<T>`
`flow.collect { }`	`await for (final x in stream) { } / stream.listen(...)`
`MutableSharedFlow`	`StreamController.broadcast()`
`StateFlow / LiveData`	`ValueNotifier / ChangeNotifier`
`collectAsState() / observe`	`StreamBuilder / FutureBuilder`
`job.cancel()`	`subscription.cancel()`

FutureBuilder: loading / error / data in one widget

lib/features/components/network_demo.dart (simplified)

```
FutureBuilder<List<Post>>(
  future: _future,
  builder: (context, snapshot) {
    if (snapshot.connectionState ==
        ConnectionState.waiting) {
      return const CircularProgressIndicator();
    }
    if (snapshot.hasError) {
      return Text('Error: ${snapshot.error}');
    }
    final posts = snapshot.data!;
    return ListView.builder(...);
  },
)
```

← ≈ observing LiveData>
← create it in initState, NOT in build!
← Resource.Loading
← Resource.Error
← Resource.Success



Watch it!

Creating the Future inside build() - `future: api.fetch()` - fires a new network request on **every rebuild**. Create it once in `initState()` (≈ ViewModel init) and store it in a field.

Isolates: real parallelism

Heavy CPU work (parsing a 5 MB JSON, image filters, crypto) on the main isolate freezes frames - that's Flutter's version of an ANR. An **Isolate** is a thread with its **own memory**; isolates share nothing and talk by passing messages. No locks, no data races.

ANDROID (Kotlin)

```
val primes = withContext(Dispatchers.Default) {
    countPrimes(5_000_000)
}
resultView.text = "$primes"
```

FLUTTER (Dart)

```
final primes = await Isolate.run(
  () => countPrimes(5000000),
);
setState(() => _result = primes);
```

Need	Flutter	Android
Short CPU task off the UI thread	Isolate.run(...) or compute(fn, arg)	Dispatchers.Default
Long-lived worker with messages	Isolate.spawn + SendPort/ReceivePort	HandlerThread + Handler
Delayed / repeating work while app is open	Timer, Future.delayed, Stream.periodic	Handler.postDelayed
Deferrable work even if app is closed	workmanager plugin	WorkManager
Ongoing user-visible task (music, tracking)	foreground service plugin	Foreground Service
Push messages	firebase_messaging	FirebaseMessagingService



RELAX

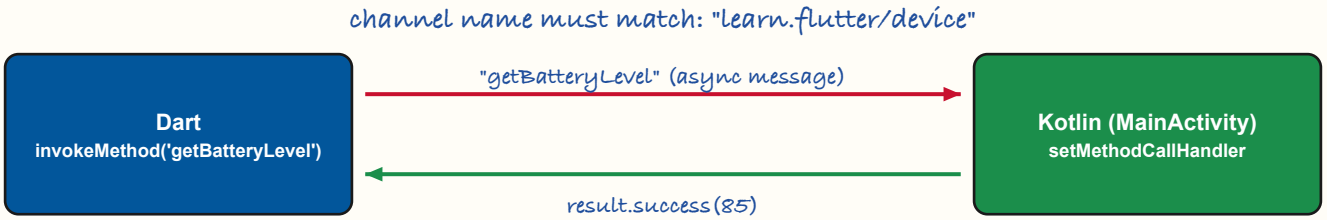
Try it: app → Learn → "Isolates & Background Work" → Animate. Tap "Run on UI isolate" and watch the spinning logo freeze. Then tap "Isolate.run()" - it keeps spinning.

CHAPTER 7

Talking to Android: Platform Channels

When Dart needs Kotlin

Dart can't call `BatteryManager` directly. A **MethodChannel** is a named, asynchronous message pipe between Dart and your Kotlin code in `MainActivity`. Arguments and results are serialised automatically (numbers, strings, lists, maps, byte arrays).



ANDROID (Kotlin)

```
// MainActivity.kt
MethodChannel(
  flutterEngine.dartExecutor.binaryMessenger,
  "learn.flutter/device"
).setMethodCallHandler { call, result ->
  when (call.method) {
    "getBatteryLevel" ->
      result.success(batteryLevel())
    else -> result.notImplemented()
  }
}
```

FLUTTER (Dart)

```
// platform_channel_demo.dart
const channel =
    MethodChannel('learn.flutter/device');

try {
  final level = await channel
      .invokeMethod<int>('getBatteryLevel');
} on PlatformException catch (e) {
  // native side called result.error(...)
} on MissingPluginException {
  // no handler (e.g. running on iOS)
}
```

Tool	What it does	Android analogy
MethodChannel	One request → one reply	Calling an SDK method
EventChannel	Native pushes a stream of events to Dart	BroadcastReceiver / sensor listener → Flow
BasicMessageChannel	Raw messages both ways	Custom IPC
Pigeon (package)	Generates type-safe channel code from a Dart interface	AIDL / generated binders
Plugins on pub.dev	Packaged channels + native code (camera, location...)	A Gradle library



BRAIN POWER

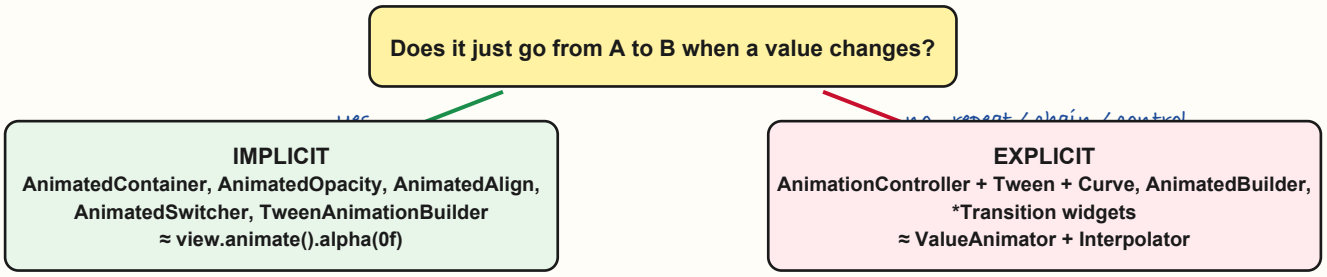
You already have a mature Android SDK written in Kotlin. What's the least-effort way to use it from a new Flutter UI? (Answer in the solutions.)

CHAPTER 8

Animations

Implicit, explicit, and Hero

Flutter renders every frame itself at 60/120 fps, so animations are first-class and cheap. There are two families. Pick the simplest one that works:



ANDROID - ViewPropertyAnimator

```
view.animate()
    .alpha(0f)
    .scaleX(1.5f)
    .setDuration(400)
    .setInterpolator(
        AccelerateDecelerateInterpolator())
    .start()
```

FLUTTER - implicit

```
AnimatedOpacity(
  opacity: _visible ? 1 : 0,
  duration: const Duration(milliseconds: 400),
  curve: Curves.easeInOut,
  child: const FlutterLogo(),
)
// later:
setState(() => _visible = false);
```

EXPLICIT ANIMATION

```
class _S extends State<Spin>
    with SingleTickerProviderStateMixin {
  late final AnimationController _c =
      AnimationController(vsync: this,
          duration: const Duration(seconds: 2));

  @override
  void dispose() { _c.dispose(); super.dispose(); }

  @override
  Widget build(BuildContext context) =>
      RotationTransition(turns: _c, child: logo);
}
// _c.forward(); _c.reverse(); _c.repeat();
```

← vsync ≈ Choreographer frames
← ≈ ValueAnimator.ofFloat(0f, 1f)
← ALWAYS dispose ≈ cancel()
← rebuilds only this on each tick
← start() / reverse() / repeat

Android	Flutter
ValueAnimator	`AnimationController`
TypeEvaluator	`Tween<T>` (`ColorTween`, `Tween<Offset>`...)
Interpolator	`Curve` (`Curves.easeOut`, `elasticOut`, `bounceOut`...)
AnimatorSet (sequence)	`Interval` curves / `TweenSequence`

Android	Flutter
Shared element transition	`Hero(tag: ...)` on both screens
overridePendingTransition	`PageRouteBuilder(transitionsBuilder:)`
TransitionManager.beginDelayedTransition	`AnimatedSize` / `AnimatedSwitcher` / `AnimatedCrossFade`
Lottie / AnimatedVectorDrawable	`lottie` / `rive` packages

CHAPTER 9

Theming & Resources

Where did res/values go?

There's no `res/` folder in Flutter. Colours, dimensions and styles are Dart objects, and the theme flows down the widget tree.

ANDROID (XML)

```
<!-- res/values/themes.xml -->
<style name="Theme.App"
    parent="Theme.Material3.DayNight">
  <item name="colorPrimary">
      @color/indigo</item>
</style>

<!-- in a layout -->
android:textColor="?attr/colorPrimary"
```

FLUTTER (Dart)

```
MaterialApp(
  theme: ThemeData(
    colorSchemeSeed: Colors.indigo),
  darkTheme: ThemeData(
    colorSchemeSeed: Colors.indigo,
    brightness: Brightness.dark),
  themeMode: ThemeMode.system,
)

// anywhere below:
color: Theme.of(context).colorScheme.primary
```

Android	Flutter
colors.xml / themes.xml	`ThemeData + ColorScheme (lib/core/theme/app_theme.dart)`
values-night/	`darkTheme + themeMode`
?attr/colorPrimary	`Theme.of(context).colorScheme.primary`
TextAppearance.Material3.TitleLarge	`Theme.of(context).textTheme.titleLarge`
strings.xml, values-fr/strings.xml	`lib/l10n/app_en.arb, app_fr.arb + flutter gen-l10n`
getString(R.string.hello)	`AppLocalizations.of(context)!.hello`
drawable-mdpi / xhdpi / xxhdpi	`assets/images/, 2.0x/, 3.0x/ folders`
R.drawable.logo	`Image.asset('assets/images/logo.png')`
Vector drawables	`flutter_svg package, or Icons.* (Material icons font)`
res/font/ + fontFamily	`fonts: in pubspec.yaml, or google_fonts package`
dp	`logical pixels - the default unit everywhere`
sp	`fontSize × MediaQuery.textScalerOf(context) (applied automatically)`
Configuration / DisplayMetrics	`MediaQuery.sizeOf(context), orientationOf, paddingOf...`

there are no **Dumb Questions**

Q: **How does dark mode work in this app?**

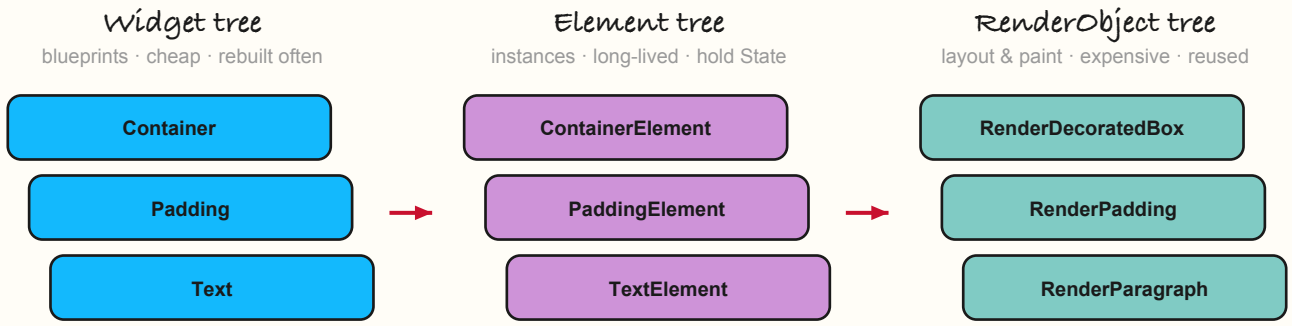
A: `ThemeProvider` (lib/state) stores a `ThemeMode`, persists it in SharedPreferences and calls notifyListeners(). `app.dart` watches it and passes it to MaterialApp. Change it in the Settings tab ≈ `AppCompatDelegate.setDefaultNightMode()`.

CHAPTER 10

Under the Hood

Widget, Element & RenderObject trees, Keys and BuildContext

If build() creates brand-new widgets every time, why is Flutter fast? Because there are **three** trees, and only the cheap one is thrown away.



same type + same key ⇒ Element & RenderObject are UPDATED, not recreated

Thing	Job	Android analogy
Widget	Immutable config. Your build() returns these.	The XML definition of a view
Element	The live instance at a spot in the tree. Holds State. Decides update vs replace.	The View object instance
RenderObject	Does measure, layout, paint and hit testing.	onMeasure / onLayout / onDraw
Key	Tells Elements which new widget matches which old one.	Stable IDs in RecyclerView
BuildContext	A handle to the Element - "where am I in the tree?"	Context (but tree-scoped)

BuildContext: your address in the tree

`Theme.of(context)`, `Navigator.of(context)`, `MediaQuery.of(context)` and `context.watch<T>()` all **walk up** the tree from your position to the nearest ancestor of that type. That's why the context you use matters.



Watch it!

After an `await`, the widget may have been removed (user pressed Back). Using its context then is like touching a destroyed Activity. Always check: `if (!context.mounted) return;` (or `if (!mounted) return;` inside a State). The linter rule *use_build_context_synchronously* will nag you - listen to it.



SHARPEN YOUR PENCIL

True or false?

Statement	T / F
Widgets are mutable and updated in place.	
A State object survives when its parent rebuilds.	
Changing a Text to an Icon at the same position reuses the Element.	
context.read is the right choice inside onPressed.	
I/O inside async code blocks the UI thread.	



BULLET POINTS

- Declarative UI: describe the UI for the current state; setState to change it.
- Row/Column/Stack/Expanded cover most layouts. Constraints down, sizes up.
- ListView.builder is RecyclerView without the Adapter.
- Navigator.push/pop ≈ startActivity/finish; await push() for results.
- ChangeNotifier ≈ ViewModel; watch in build, read in callbacks.
- Future ≈ suspend, Stream ≈ Flow, Isolate.run ≈ Dispatchers.Default.
- MethodChannel lets Dart call your Kotlin code.
- Implicit animations first; AnimationController when you need control.
- Three trees make rebuilding cheap; Keys help match list items.

Exercise Solutions

Chapter 2 - the list row

```
Row(children: [
  const CircleAvatar(child: Icon(Icons.person)),
  const SizedBox(width: 12),
  Expanded(                          // takes the remaining width (weight = 1)
    child: Column(
      crossAxisAlignment: CrossAxisAlignment.start,
      children: [
        Text(name, style: const TextStyle(fontWeight: FontWeight.bold)),
        Text(subtitle),
      ],
    ),
  ),
  const Icon(Icons.chevron_right),
])
// ...or simply: ListTile(leading: ..., title: ..., subtitle: ..., trailing: ...)
```

Chapter 7 - reuse your Android SDK

Keep the SDK in the `android/` module (Gradle dependency), and expose the handful of calls the UI needs through a MethodChannel (or Pigeon for type safety) in MainActivity. Later you can package it as a Flutter plugin.

Chapter 10 - true or false

Statement	Answer
Widgets are mutable and updated in place.	FALSE - they are immutable and recreated.
A State object survives when its parent rebuilds.	TRUE - the Element keeps it.
Changing a Text to an Icon reuses the Element.	FALSE - different type ⇒ new Element.
context.read is the right choice inside onPressed.	TRUE.
I/O inside async code blocks the UI thread.	FALSE - it runs outside Dart and returns as an event.